

Question 2 Exam Notes — Linear & Logistic Regression

0. FIRST DECISION IN THE EXAM

Look at the target `y`.

- Continuous number (salary, price, electricity use, time) -> Linear Regression
- Binary class (0/1, fraud/not fraud, disease/no disease) -> Logistic Regression

Do not choose the model from the feature names. Choose it from the target.

PART A — LINEAR REGRESSION

1. Core pipeline

Continuous target -> X/y -> split -> optional standardization -> fit -> predict -> requested error/plot/baseline/CI

Load and create X/y

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("data.csv")

features = ["feature1", "feature2", "feature3"]
target = "target"

X = df[features].to_numpy(dtype=float)
y = df[target].to_numpy(dtype=float)
```

2. First 80% train / last 20% test

If the question literally says first 80% and last 20%, use slicing.

```
split = int(0.8 * len(X))

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]
```

Exam wording

- fitting / learning observations = TRAIN
- held-out / unseen / evaluation observations = TEST

3. Standardization — TRAIN statistics only

If asked to standardize without using test information:

```
mu = np.mean(X_train, axis=0)
sd = np.std(X_train, axis=0)

sd[sd == 0] = 1.0

X_train_std = (X_train - mu) / sd
```

```
X_test_std = (X_test - mu) / sd
```

Important: Never calculate separate test mean/std.

If you standardize, fit and predict using `X\_train\_std` and `X\_test\_std`.

4. Fit Linear Regression

Without standardization:

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

With standardization:

```
model = LinearRegression()
model.fit(X_train_std, y_train)
y_pred = model.predict(X_test_std)
```

5. Residual / MAE / MSE / MARE

```
# Residuals
residuals = y_test - y_pred

# Mean Absolute Error
mae = np.mean(np.abs(y_test - y_pred))

# Mean Squared Error
mse = np.mean((y_test - y_pred)**2)

# Mean Absolute Relative Error
mare = np.mean(np.abs((y_test - y_pred) / y_test))
```

Recognition

- residual -> `y\_true - y\_pred`
- absolute -> `np.abs`
- squared -> `2`
- relative -> divide by true value

6. Training-mean baseline

If the question says:

> constant predictor / naive predictor / average response of fitting observations

use mean of `y\_train`, not `y\_test`.

```
baseline_value = np.mean(y_train)

baseline_pred = np.full(
    len(y_test),
    baseline_value,
    dtype=float
)

model_mse = np.mean((y_test - y_pred)**2)
baseline_mse = np.mean((y_test - baseline_pred)**2)

print("Model MSE:", model_mse)
print("Baseline MSE:", baseline_mse)

if model_mse < baseline_mse:
```

```

    print("Regression model is better")
else:
    print("Baseline is better")

```

Smaller MSE = better.

7. Predicted-vs-true scatter plot

Remember `plt.scatter(x, y)`.

Predicted on x-axis, true on y-axis:

```

plt.scatter(y_pred, y_test)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()

```

True on x-axis, predicted on y-axis:

```

plt.scatter(y_test, y_pred)

```

PART B — DKW FOR RESIDUAL ECDF

Use DKW when the question asks for:

- ECDF / EDF / CDF
- cumulative distribution
- confidence bands for a distribution function

Example: 99% DKW bands for residual ECDF.

```

residuals = y_test - y_pred
sorted_residuals = np.sort(residuals)

n = len(sorted_residuals)
F_n = np.arange(1, n + 1) / n

delta = 0.01 # 99%

epsilon = np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = np.maximum(F_n - epsilon, 0)
upper = np.minimum(F_n + epsilon, 1)

plt.step(sorted_residuals, F_n, where="post")
plt.step(sorted_residuals, lower, where="post")
plt.step(sorted_residuals, upper, where="post")
plt.show()

```

DKW recognition rule

ECDF / CDF / distribution confidence band -> DKW

PART C — LOGISTIC REGRESSION

8. Core recognition

Use Logistic Regression when `y` is binary:

- fraud = 0/1
- disease = 0/1
- spam = 0/1

```
from sklearn.linear_model import LogisticRegression
```

Class vs probability

```
# Direct class prediction
pred = model.predict(X_test)

# Probability of class 1
prob = model.predict_proba(X_test)[:, 1]
```

`[:, 1]` = all observations, probability of class 1.

9. 60% train / 20% validation / 20% test

If question says first 60%, next 20%, last 20%:

```
n = len(X)

train_end = int(0.60 * n)
val_end   = int(0.80 * n)

X_train = X[:train_end]
y_train = y[:train_end]

X_val = X[train_end:val_end]
y_val = y[train_end:val_end]

X_test = X[val_end:]
y_test = y[val_end:]
```

Why `0.80`?

Training ends at 60%; validation is the next 20%, so validation ends at 80%.

Train builds -> Validation chooses -> Test judges.

10. Full logistic beginning

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("transactions.csv")

features = ["amount", "account_age", "transactions_24h"]
target = "fraud"

X = df[features].to_numpy(dtype=float)
y = df[target].to_numpy(dtype=int)

n = len(X)
train_end = int(0.60 * n)
val_end = int(0.80 * n)

X_train = X[:train_end]
y_train = y[:train_end]

X_val = X[train_end:val_end]
y_val = y[train_end:val_end]

X_test = X[val_end:]
y_test = y[val_end:]

model = LogisticRegression()
model.fit(X_train, y_train)
```

```
prob_val = model.predict_proba(X_val)[: , 1]
prob_test = model.predict_proba(X_test)[: , 1]
```

11. Given threshold

If class 1 when probability is at least `t`:

```
pred = (prob >= t).astype(int)
```

Example threshold 0.65:

```
pred = (prob >= 0.65).astype(int)
```

probability >= threshold -> class 1

probability < threshold -> class 0

12. TP / TN / FP / FN

```
TP = np.sum((y_true == 1) & (pred == 1))
TN = np.sum((y_true == 0) & (pred == 0))
FP = np.sum((y_true == 0) & (pred == 1))
FN = np.sum((y_true == 1) & (pred == 0))
```

Memory table:

Actual	Predicted	Result
---	---	---
1	1	TP
0	0	TN
0	1	FP
1	0	FN

PART C1 — EXACT PRACTICE QUESTION: FRAUD LOGISTIC REGRESSION

Question

A bank has a dataset called `transactions.csv` with the following columns:

- `amount`
- `account\_age`
- `transactions\_24h`
- `fraud` — where `0` means legitimate and `1` means fraud

Use the first 60% of the observations for training, the next 20% for validation, and the last 20% for testing. Fit a suitable model for predicting fraud. The validation set will later be used to select a classification threshold.

Exact clean code from the start

```
import numpy as np
```

```

import pandas as pd

from sklearn.linear_model import LogisticRegression

# =====
# 1. LOAD DATA
# =====

df = pd.read_csv("transactions.csv")

# =====
# 2. CREATE X AND y
# =====

features = [
    "amount",
    "account_age",
    "transactions_24h"
]

target = "fraud"

X = df[features].to_numpy(dtype=float)
y = df[target].to_numpy(dtype=int)

# =====
# 3. SPLIT:
# FIRST 60% TRAIN
# NEXT 20% VALIDATION
# LAST 20% TEST
# =====

n = len(X)

train_end = int(0.60 * n)
val_end   = int(0.80 * n)

# TRAIN: 0% -> 60%
X_train = X[:train_end]
y_train = y[:train_end]

# VALIDATION: 60% -> 80%
X_val = X[train_end:val_end]
y_val = y[train_end:val_end]

# TEST: 80% -> 100%
X_test = X[val_end:]
y_test = y[val_end:]

# =====
# 4. FIT LOGISTIC REGRESSION ON TRAIN ONLY
# =====

model = LogisticRegression()

model.fit(
    X_train,
    y_train
)

# =====
# 5. GET PROBABILITY OF FRAUD = CLASS 1
# =====

prob_val = model.predict_proba(X_val)[:, 1]

prob_test = model.predict_proba(X_test)[:, 1]

```

At this point:

```

TRAIN
-> used to fit LogisticRegression

VALIDATION
-> prob_val

```

-> later used to choose best threshold

TEST

-> prob_test

-> later used for final evaluation

The important structure is:

CSV

-> X/y

-> 60/20/20 split

-> fit Logistic Regression on TRAIN

-> predict_proba on VALIDATION

-> choose threshold

-> apply chosen threshold to TEST

-> final loss/cost/precision/recall if requested

Why is `val_end = int(0.80 * n)`?

Because:

60% training + next 20% validation = 80%

So validation starts at the 60% position and ends at the 80% position.

PART D — COST-SENSITIVE THRESHOLD

13. Reusable cost function

Change the cost numbers to match the question.

```
def cost(y_true, y_prob, threshold):
    pred = (y_prob >= threshold).astype(int)

    TP = np.sum((y_true == 1) & (pred == 1))
    TN = np.sum((y_true == 0) & (pred == 0))
    FP = np.sum((y_true == 0) & (pred == 1))
    FN = np.sum((y_true == 1) & (pred == 0))

    # CHANGE THESE COSTS
    total = 100*TP + 0*TN + 120*FP + 600*FN

    return float(total / len(y_true))
```

Generic form:

```
total = (
    TP_cost * TP
    + TN_cost * TN
    + FP_cost * FP
    + FN_cost * FN
)
```

14. Search best threshold on VALIDATION

```
thresholds = np.arange(0.00, 1.01, 0.01)

costs = np.array([
    cost(y_val, prob_val, t)
    for t in thresholds
])

best_idx = np.argmin(costs)
best_threshold = thresholds[best_idx]
best_cost_val = costs[best_idx]
```

Why `argmin`?

Cost is better when it is smaller.

Pipeline:

validation probabilities -> threshold -> predictions -> TP/TN/FP/FN -> cost -> repeat -> argmin -> best threshold

15. Apply chosen threshold to TEST

Do not choose a new threshold on test.

```
pred_test = (prob_test >= best_threshold).astype(int)

test_cost = cost(
    y_test,
    prob_test,
    best_threshold
)
```

Validation chooses threshold. Test only uses it.

PART E — PRECISION, RECALL, 0-1 LOSS

These are OPTIONAL branches. Calculate them only when the question asks.

16. Precision and Recall

```
TP = np.sum((y_test == 1) & (pred_test == 1))
FP = np.sum((y_test == 0) & (pred_test == 1))
FN = np.sum((y_test == 1) & (pred_test == 0))

precision = TP / (TP + FP)
recall = TP / (TP + FN)
```

Memory:

- Precision -> FP
- Recall -> FN

Precision asks: of predicted positives, how many were really positive?

Recall asks: of real positives, how many did we catch?

17. Accuracy and 0-1 loss

```
accuracy = np.mean(pred_test == y_test)
loss_01 = np.mean(pred_test != y_test)
```

- Accuracy: fraction correct -> higher better
- 0-1 loss: fraction wrong -> lower better

If choosing threshold to minimize 0-1 loss:

```
thresholds = np.arange(0.00, 1.01, 0.01)
losses = []

for t in thresholds:
    pred_val = (prob_val >= t).astype(int)
    losses.append(np.mean(pred_val != y_val))

losses = np.array(losses)
```



```
best_idx = np.argmin(losses)
best_threshold = thresholds[best_idx]
```

PART F — Hoeffding

Use Hoeffding when the question asks for a confidence interval for one average/expected bounded quantity:

- average 0-1 loss
- expected classification error
- average capped regression error
- average bounded business cost

18. 0-1 loss Hoeffding CI

0-1 loss is bounded in $[0,1]$.

```
losses = (pred_test != y_test).astype(float)

mean_loss = np.mean(losses)
n = len(losses)

delta = 0.01 # 99%

epsilon = np.sqrt(
    np.log(2 / delta) / (2 * n)
)

interval = (
    max(0.0, mean_loss - epsilon),
    min(1.0, mean_loss + epsilon)
)
```

Confidence conversion:

- 95% -> $\delta = 0.05$
- 99% -> $\delta = 0.01$
- 90% -> $\delta = 0.10$

19. Hoeffding with capped regression error

If absolute errors are capped at 80:

```
errors = np.abs(y_test - y_pred)
clipped_errors = np.minimum(errors, 80)

mean_error = np.mean(clipped_errors)
n = len(clipped_errors)

delta = 0.01

epsilon = 80 * np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = mean_error - epsilon
upper = mean_error + epsilon
```

20. Hoeffding with business cost

Hoeffding uses:

RANGE = maximum possible value - minimum possible value

If possible costs are `[20, 50, 100, 700]`:

$$\text{RANGE} = 700 - 20$$

Then:

```
epsilon = RANGE * np.sqrt(
    np.log(2 / delta) / (2 * n)
)
```

If costs range from 0 to 800, `RANGE = 800`.

PART G — Hoeffding VS DKW

This distinction is important.

Hoeffding

Question says:

- expected
- mean
- average loss
- average cost
- average bounded error
- confidence interval for one number

-> Hoeffding

DKW

Question says:

- ECDF / EDF
- CDF
- cumulative distribution
- distribution function
- confidence bands

-> DKW

Memory:

MEAN -> Hoeffding

CDF -> DKW

PART H — CALIBRATION BRANCH (LOWER PRIORITY)

If the question explicitly asks to calibrate model outputs using a separate calibration sample:

```

from sklearn.tree import DecisionTreeRegressor

# Original classifier already fitted on training data

raw_calib = model.predict(X_calib).reshape(-1, 1)

calibrator = DecisionTreeRegressor()
calibrator.fit(raw_calib, y_calib)

raw_test = model.predict(X_test).reshape(-1, 1)
calibrated_prob = calibrator.predict(raw_test)

pred = (calibrated_prob >= 0.5).astype(int)

loss_01 = np.mean(pred != y_test)

```

Recognition:

original classifier -> raw score -> calibration model -> calibrated probability -> threshold -> class -> evaluate

Use this only when the question explicitly gives a calibration setup.

PART I — EXAM DECISION TREE

Step 1: Look at target

Continuous? -> Linear Regression

0/1? -> Logistic Regression

Step 2: Read the final subquestion

Linear

- residuals -> ``y_test - y_pred``
- MAE -> absolute
- MSE -> squared
- MARE -> absolute relative
- baseline -> ``mean(y_train)``
- ECDF/CDF bands -> DKW
- average bounded error CI -> Hoeffding

Logistic

- probability -> ``predict_proba()[,1]``
- given threshold -> ``(prob >= t).astype(int)``
- minimum business cost -> validation threshold search + ``argmin(costs)``
- minimum 0-1 loss -> validation threshold search + ``argmin(losses)``
- precision -> ``TP/(TP+FP)``
- recall -> ``TP/(TP+FN)``
- 0-1 loss -> ``mean(pred != y)``
- average bounded loss/cost CI -> Hoeffding
- calibration explicitly requested -> calibration branch

PART J — COMMON EXAM MISTAKES

1. Using test statistics for standardization.
 - WRONG: calculate mean/std separately on test.
 - RIGHT: train mean/std used on both.
2. Using `mean(y_test)` for baseline.
 - RIGHT: `mean(y_train)`.
3. Confusing MAE and MSE.
 - MAE -> `abs`
 - MSE -> `^2`
4. Confusing residual and absolute error.
 - residual -> `y_test - y_pred`
5. Using `predict()` when probability is requested.
 - probability -> `predict_proba()[0,1]`
6. Reversing threshold.
 - class 1 if `prob >= threshold`
7. Choosing threshold on test.
 - validation chooses; test judges.
8. Using `argmax` for cost/loss.
 - lower is better -> `argmin`
9. Precision/recall parentheses.
 - `TP / (TP + FP)`
 - `TP / (TP + FN)`
10. Mixing DKW and Hoeffding.
 - mean -> Hoeffding
 - CDF -> DKW
11. If the question says first/next/last, do not randomly shuffle unless instructed.
12. If you transformed X, remember to fit/predict using the transformed X.

PART K — LAST-MINUTE CHEAT SHEET

```
# LINEAR
model = LinearRegression()
model.fit(X_train_std, y_train)
y_pred = model.predict(X_test_std)

residuals = y_test - y_pred
mae = np.mean(np.abs(y_test - y_pred))
mse = np.mean((y_test - y_pred)**2)
mare = np.mean(np.abs((y_test - y_pred) / y_test))

baseline = np.mean(y_train)
```

```

baseline_pred = np.full(len(y_test), baseline)
baseline_mse = np.mean((y_test - baseline_pred)**2)

# LOGISTIC
model = LogisticRegression()
model.fit(X_train, y_train)

prob_val = model.predict_proba(X_val)[:, 1]
prob_test = model.predict_proba(X_test)[:, 1]

pred_test = (prob_test >= best_threshold).astype(int)

loss_01 = np.mean(pred_test != y_test)

# CONFUSION COUNTS
TP = np.sum((y == 1) & (pred == 1))
TN = np.sum((y == 0) & (pred == 0))
FP = np.sum((y == 0) & (pred == 1))
FN = np.sum((y == 1) & (pred == 0))

precision = TP / (TP + FP)
recall = TP / (TP + FN)

# Hoeffding
epsilon = RANGE * np.sqrt(
    np.log(2 / delta) / (2 * n)
)

# DKW
epsilon = np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = np.maximum(F_n - epsilon, 0)
upper = np.minimum(F_n + epsilon, 1)

```

FINAL MEMORY MAP

Continuous y -> LINEAR

`split -> train-only standardize if asked -> fit -> predict -> requested metric`

Binary y -> LOGISTIC

`fit train -> probabilities -> validation chooses threshold -> test evaluates`

Mean/expected bounded quantity -> Hoeffding

CDF/ECDF confidence bands -> DKW

Lower cost/loss is better -> ARGMIN

Train builds -> Validation chooses -> Test judges